Assignment 2: Architectural Choices, Tokenizers, Decoding, and Fine Tuning

Lior Alafi 032543472, Tal Somech 315473033

23 May 2026

Architectural Choices

Architecture Extraction Method

We've extracted the architectural information from three complementary sources: the generated `architecture.csv` file, the generated `arch.md` file, and the official HuggingFace model cards.

The `architecture.csv` file was generated directly from each model's HuggingFace `config.json`. Therefore, the fields in the CSV reflect configuration-level scalar hyperparameters such as `hidden_size`, `num_hidden_layers`, `num_attention_heads`, `num_key_value_heads`, `intermediate_size`, `hidden_act`, `rms_norm_eps`, `max_position_embeddings`, `vocab_size`, and, when relevant, Mixture-of-Experts fields such as the number of experts and the number of experts selected per token.

The `arch.md` file was generated by instantiating each model from its configuration under `init_empty_weights`. This allowed us to inspect the model structure without loading the full checkpoint weights into memory. We then printed the module names and parameter shapes. This gave a lower-level view of the actual architecture, including the shapes of the embedding matrix, the attention projections, the MLP projections, normalization layers, MoE expert matrices, and the final `lm_head`.

Finally, we compared the extracted information with the corresponding model cards. The model cards are useful because they often describe the intended or advertised model behavior, such as long-context support, training data, special tokenizer changes, or high-level architectural design choices. However, model cards sometimes report effective or advertised context length, while the configuration file reports a lower-level field such as `max_position_embeddings`. For this reason, We treated the config, the architecture dump, and the model card as related but not always identical sources.

Uncertainties, Missing Fields, and Source Disagreements

Most scalar fields in `architecture.csv` agree with the architecture dump in `arch.md`. For example, for Llama-3.1-8B-Instruct and Mistral-7B-Instruct-v0.3, the CSV reports `hidden_size=4096`, `num_layers=32`, `num_attention_heads=32`, `num_kv_heads=8`, and `mlp_size=14336`. The architecture dump then confirms the corresponding projection shapes: the query projection has shape 4096×4096, while the key and value projections have shape 1024×4096. This matches grouped-query attention, since $8 \cdot 128 = 1024$.

The main disagreements are related to context length. In the CSV, the context length was taken from the downloaded config, usually from the field `max_position_embeddings`. This is a consistent and reproducible choice, but it does not always match the value advertised in the model card. The clearest example is Qwen2.5-7B-Instruct. In our config, Qwen

has `max_position_embeddings=32768`, so the CSV reports a context length of 32768. However, the model card advertises full 131,072-token context support. The config also contains `sliding_window=131072` but `use_sliding_window=false`, so the effective long-context behavior appears to depend on deployment and runtime settings, not only on the `max_position_embeddings` field.

DeepSeek-V3 has a similar but opposite-looking discrepancy. The CSV reports `context_length=16384` because that is the value of `max_position_embeddings` in the config. The model card, however, discusses evaluation and support up to 128K context. Therefore, We distinguish between the config-level maximum position field and the advertised or evaluated context window.

Another source of missing detail is the limited schema of the CSV. The CSV is a compact summary table, so it cannot represent every architectural detail. For example, OLMo-2 contains `q_norm` and `k_norm` inside the attention block. Phi-4-mini uses a fused `qkv_proj` instead of separate `q_proj`, `k_proj`, and `v_proj` modules. DeepSeek-V3 uses a non-standard MLA-style attention mechanism and a complex MoE structure. Granite includes special scaling multipliers. These details appear in `arch.md` or in the config, but they are not represented by the fixed CSV columns.

Architectural Differences Not Directly Represented in the CSV

Category	**Relevant models**	**Detail not shown directly in the CSV**
GQA vs. standard MHA	Llama, Mistral, Qwen, Granite, Phi, Falcon, and Dicta use GQA. OLMo and SmolLM2 use standard MHA.	The CSV includes the number of attention heads and KV heads, but it does not explicitly label the attention type as grouped-query attention (GQA) or standard multi-head attention (MHA).
Non-standard attention mechanism	DeepSeek-V3	DeepSeek-V3 uses MLA-style attention with low-rank Q/KV projections, rather than ordinary separate Q, K, and V projections.
Fused QKV projection	Phi-4-mini	Phi-4-mini uses a fused qkv_proj module instead of separate q_proj, k_proj, and v_proj modules.
Fused MLP projection	Phi-4-mini	Phi-4-mini uses gate_up_proj instead of separate gate_proj and up_proj matrices in the MLP block.
Q/K normalization	OLMo-2	OLMo-2 includes q_norm and k_norm inside the attention block. The CSV only reports the general normalization type.
Internal attention normalization	DeepSeek-V3	DeepSeek-V3 includes internal normalization components such as q_a_layernorm and kv_a_layernorm as part of its attention mechanism.
Mixture-of-Experts structure	DeepSeek-V3	The CSV summarizes the MoE fields, but it does not show the expert matrix shapes or the transition from the first dense layers to later MoE layers.
Tied embeddings	Granite, SmolLM2, and Phi-4-mini	The CSV includes the vocabulary size, but it does not say that the input embedding matrix and the output LM head share weights.
Untied embeddings	Llama, Mistral, Qwen, OLMo, DeepSeek, and DictaLM2	The CSV includes the vocabulary size, but it does not say that the input embedding matrix and the output LM head are separate parameters.

Table 1: Architectural differences grouped by category.

Category	Relevant models	Detail not shown directly in the CSV
RoPE scaling details	Llama-3.1, Phi-4-mini, and DeepSeek-V3	The CSV gives a short positional-encoding label, but it does not include the full RoPE scaling hyperparameters, such as Llama-3 RoPE scaling, LongRoPE factors, or YaRN parameters.
Sliding-window or long-context fields	Qwen and DictaLM2	The CSV includes only one context-length value, but not additional config fields related to sliding-window attention or document-level attention.
Special scaling multipliers	Granite	Granite includes embedding, attention, residual, and logits scaling multipliers that are not represented in the CSV.
Large attention head dimension	Falcon3	Falcon3 has a larger attention head dimension than most of the other models in the comparison. This is obtained by dividing hidden size by the number of attention heads.
Unusually large MLP ratio	Qwen and Falcon3	Qwen has a relatively large MLP intermediate size compared to its hidden size, and Falcon3 has an especially large MLP ratio. The CSV contains the raw sizes but does not explicitly interpret this ratio.
Hebrew-adapted tokenizer	DictaLM2	The CSV gives the vocabulary size, but it does not explain that the tokenizer vocabulary was adapted or extended for Hebrew.
Model-specific implementation details	DictaLM2 and Granite	DictaLM2 is Mistral-like and adapted for Hebrew, while Granite includes implementation-specific scaling constants and tied embeddings. These details are not captured by the compact CSV columns.

Table 2: Architectural differences grouped by category.

Main Architectural Differences

Most of the models follow the same broad decoder-only Transformer pattern: causal self-attention, an MLP block, residual connections, RMSNorm, RoPE-style positional encoding, and the SiLU activation used inside a gated MLP. This is the dominant design pattern among the dense models in the comparison.

The most common attention design is grouped-query attention. Llama, Mistral, Qwen, Granite, Phi, Falcon, and DictaLM2 all use fewer KV heads than query heads. This reduces the size of the key-value cache during generation and makes inference more efficient. OLMo-2 and SmolLM2 are exceptions: in both models, the number of KV heads equals the number of query heads, so they use standard multi-head attention rather than GQA.

The strongest architectural outlier is DeepSeek-V3. Unlike the other models, it is not a standard dense 7B/8B model. It is a large sparse MoE model. It uses DeepSeekMoE with many routed experts, one shared expert, and only a subset of experts activated per token. It also uses MLA-style attention, which is not the usual separate Q/K/V projection structure. Thus, DeepSeek-V3 is different both in its feed-forward block and in its attention block.

Phi-4-mini is another implementation-level outlier. Architecturally, it is still a dense decoder-only model, but its parameter structure is fused: instead of separate Q, K, and V projections, the architecture uses a single `qkv_proj`; instead of separate `gate_proj` and `up_proj`, it uses a fused `gate_up_proj`. This does not change the high-level Transformer idea, but it changes how the architecture appears in the parameter dump and how one should interpret the shapes.

OLMo-2 is interesting because it is not unusual in size, but it has additional normalization inside the attention mechanism. In particular, the architecture dump shows `q_norm` and `k_norm`. This is not captured by a simple `norm_type=rmsnorm` field in the CSV.

Granite is also interesting because it includes model-specific scaling constants, such as embedding, attention, residual, and logits multipliers. These are not part of the standard minimal Transformer description, but they are part of the actual implementation. Granite also uses tied embeddings, unlike many of the other 7B/8B models.

Falcon3 differs in its sizing choices. Its hidden size is 3072 and it has only 12 attention heads, which gives a head dimension of $3072/12 = 256$. Most other models in this comparison use head dimension 128. Falcon3 also has a very large MLP intermediate size relative to its hidden size.

DictaLM2 is mainly interesting from the tokenizer and adaptation perspective. Its architecture is Mistral-like, but the model is adapted for Hebrew and uses an expanded vocabulary. This shows that some models differ more in tokenizer and training choices than in their core Transformer block.

General Trends and Insights

Common Choices

Several choices are almost universal in this set of models. First, all models use the SiLU activation according to the CSV. In practice, this usually appears inside a gated MLP block, often described as a SwiGLU-style feed-forward layer. Second, all models use RMSNorm rather than classical LayerNorm. Third, all models use some form of RoPE positional encoding, although the exact variant differs. Some use plain RoPE, while others use scaled versions such as Llama-3 RoPE scaling, LongRoPE, or YaRN.

Another common pattern is that many 7B/8B-class dense models use a hidden size around 4096 and around 32 layers. Llama-3.1-8B, Mistral-7B, OLMo-2-7B, and DictaLM2 all use 32 layers and hidden size 4096. Granite uses the same hidden size but increases depth to 40 layers. Qwen uses a somewhat smaller hidden size of 3584 and 28 layers, while Phi and Falcon use hidden size 3072.

Attention Trends

The dominant attention trend is the use of GQA. A useful rule of thumb is that the query projection keeps the full hidden dimension, while the key and value projections are reduced according to the number of KV heads. For example, in Mistral and Llama, the hidden size is 4096, there are 32 query heads, and there are 8 KV heads. The head dimension is $4096/32 = 128$, so the K and V projection output size is $8 \cdot 128 = 1024$. This exactly matches the architecture dump, where `k_proj` and `v_proj` have output dimension 1024.

Qwen follows the same principle even more aggressively. It has hidden size 3584, 28 query

heads, and 4 KV heads. The head dimension is $3584/28 = 128$, and the K/V projection output size is $4 \cdot 128 = 512$. Thus, Qwen has much smaller K/V projections relative to Q.

Falcon3 is unusual because its head dimension is 256 rather than 128. It has hidden size 3072 and 12 query heads, so $3072/12 = 256$. This is explicitly a different sizing choice from the common 128-dimensional head pattern.

MLP Sizing Trends

The MLP intermediate size is usually several times larger than the hidden size, but there is no single universal multiplier. Some common ratios are:

$$\frac{\text{MLP size}}{\text{hidden size}} = \frac{14336}{4096} = 3.5$$

for Llama, Mistral, and DictaLM2. OLMo-2 uses

$$\frac{11008}{4096} \approx 2.69,$$

while Granite uses

$$\frac{12800}{4096} = 3.125.$$

Qwen is larger:

$$\frac{18944}{3584} \approx 5.29,$$

and Falcon3 is the most extreme dense model in this comparison:

$$\frac{23040}{3072} = 7.5.$$

SmolLM2 uses a clean ratio of

$$\frac{8192}{2048} = 4.$$

Phi-4-mini uses

$$\frac{8192}{3072} \approx 2.67.$$

Thus, while the MLP is almost always larger than the hidden state, the exact multiplier varies significantly. There is no consensus on a single ratio. The choice seems to reflect a tradeoff between capacity, compute cost, and the specific training recipe.

Vocabulary and Embedding Trends

Vocabulary size varies substantially. Mistral has a relatively small vocabulary of 32768. DictaLM2 has a slightly larger vocabulary of 33152, consistent with Hebrew adaptation. SmolLM2 and Granite use vocabularies around 49K. Llama, Falcon, Qwen, and Phi use much larger vocabularies: Llama has 128256, Falcon has 131072, Qwen has 152064, and Phi has 200064.

This shows that vocabulary size is not tightly determined by model width. Instead, it depends heavily on tokenizer design, supported languages, special tokens, and product requirements such as function calling or multilingual coverage.

Another difference is whether the input embeddings and output LM head are tied. Granite, SmolLM2, and Phi-4-mini use tied embeddings, while Llama, Mistral, Qwen, OLMo, DeepSeek, and DictaLM2 use untied embeddings. The CSV reports the vocabulary size but does not include this distinction.

Position Encoding and Context Length

All models use RoPE or a RoPE variant, but long-context support is not handled uniformly. Llama-3.1 uses Llama-3 RoPE scaling. Phi-4-mini uses LongRoPE. DeepSeek-V3 uses YaRN. Qwen includes sliding-window-related fields in the config and advertises longer context in the model card than the raw `max_position_embeddings` value suggests.

Therefore, there is consensus on using RoPE-style positional encoding, but there is no consensus on how to extend it to long contexts. Each family appears to use its own scaling method or deployment convention.

Reflection: What Architecture Would We Choose for a New Model?

If We were building a new general-purpose dense model in the 7B–8B range, We would choose a conservative architecture close to the current dominant pattern, but we would adopt a few modern efficiency improvements.

We would use a decoder-only Transformer with approximately 32 layers and hidden size 4096. This choice is well supported by several successful models in the comparison, including Llama, Mistral, OLMo, and DictaLM2. It gives a familiar and well-tested scale for a 7B–8B model.

For attention, We would use grouped-query attention rather than standard multi-head attention. A reasonable setting would be 32 query heads and 8 KV heads, with head dimension 128. This gives full query capacity while reducing KV-cache memory during inference. The resulting projection sizes would be:

$$d_{\text{head}} = \frac{4096}{32} = 128,$$

$$d_{\text{KV output}} = 8 \cdot 128 = 1024.$$

Thus, Q would project from 4096 to 4096, while K and V would project from 4096 to 1024. This follows the same efficient pattern used by Llama and Mistral.

For the MLP, We would use a SwiGLU-style block with SiLU activation and an intermediate size around 14336, giving an MLP-to-hidden ratio of 3.5. This ratio is common in Llama/Mistral-style models and seems like a good balance between capacity and compute. We would probably avoid an extremely large MLP ratio such as Falcon3's 7.5 unless there were strong empirical reasons, because it would increase compute cost substantially.

For normalization, We would use RMSNorm in a pre-norm layout and would also consider adding Q/K normalization, as in OLMo-2, because it is a relatively localized architectural change that may improve training stability. However, We would treat Q/K normalization as an empirical choice rather than a mandatory component.

For positional encoding, We would use RoPE with a long-context scaling method. If the target model needs robust long-context behavior, We would explicitly choose a method such as Llama-3-style RoPE scaling or LongRoPE, and we would make the relationship between `max_position_embeddings`, model-card context length, and runtime settings clear. One lesson from the comparison is that context length can become confusing when the config and model card describe different notions of context support.

For vocabulary, We would choose a tokenizer based on the target language coverage. For an English-only or mostly English model, a vocabulary around 50K–130K may be enough. For a multilingual model, especially one that needs strong Hebrew support, We would prefer a larger or adapted tokenizer. DictaLM2 is a useful example showing that tokenizer adaptation can be an important architectural and practical choice even when the core Transformer block remains Mistral-like.

Finally, We would keep the model dense unless the target scale is much larger. DeepSeek-V3 shows that MoE can be very powerful, but it also introduces major implementation complexity: expert routing, load balancing, sparse activation, and more complex inference. For a 7B–8B model, a dense GQA Transformer is a simpler and more reliable design. For a much larger model, We would consider a DeepSeek-style MoE design with a small number of active experts per token.

Overall, our proposed architecture would be:

- Decoder-only Transformer.
- 32 layers.
- Hidden size 4096.
- 32 query heads and 8 KV heads.
- Head dimension 128.
- GQA attention.
- SwiGLU-style MLP with SiLU activation.
- MLP intermediate size 14336.
- RMSNorm with pre-norm layout.
- Optional Q/K normalization after empirical testing.
- RoPE with explicit long-context scaling.
- Context target of at least 32K, preferably 128K if long-context data and evaluation are available.
- Vocabulary chosen according to language coverage, with tokenizer adaptation for Hebrew or multilingual support.
- Dense architecture for 7B–8B scale; MoE only for substantially larger models.

“‘

Tokenizers

0.1 Tokenizer Analysis Methodology

To analyze the tokenizers, we loaded each tokenizer using the Hugging Face `AutoTokenizer` interface and inspected its configuration and vocabulary. This allowed us to identify the tokenizer type, vocabulary size, special tokens, word-boundary conventions, and other relevant properties required for the `tokenizers.csv` file.

To estimate the average number of tokens per word, we collected approximately 100 English sentences and 100 Hebrew sentences. Each sentence was tokenized using the corresponding tokenizer, and the total number of produced tokens was recorded. The average tokens-per-word value was then computed as:

$$\text{Average Tokens Per Word} = \frac{\text{Total Number of Tokens}}{\text{Total Number of Words}} \tag{1}$$

Words were counted using a regular-expression-based tokenizer that separates text according to whitespace and punctuation while preserving valid word boundaries.

The average-tokens-per-word metric provides an indication of how efficiently a tokenizer represents text. Lower values indicate that words are more frequently represented as complete tokens, resulting in shorter token sequences. Higher values suggest that words are often decomposed into multiple subword units, increasing the number of tokens required to represent the same text. This metric is particularly useful when comparing tokenization efficiency across languages, as languages with richer morphology or lower representation in the tokenizer vocabulary often require more tokens per word.

In our results, English generally required fewer tokens per word than Hebrew across most models. This suggests that the tokenizers' vocabularies provide better coverage of English words, allowing them to be represented as complete tokens more frequently. Hebrew words, on the other hand, were more often split into multiple subword units, resulting in higher average token counts. This behavior is expected because most of the evaluated models were primarily trained on English-centric corpora and therefore allocate a larger portion of their vocabulary to English text.

0.2 Tokenization Comparison Across Models

To compare the tokenization behavior of different models, the sentence:

"The quick brown fox jumps over the lazy dog."

was tokenized using all ten model tokenizers. The results naturally formed three distinct groups.

Group 1 Models:

- Llama-3.1-8B-Instruct
- Qwen2.5-7B-Instruct
- OLMo-2-1124-7B-Instruct
- DeepSeek-V3
- SmolLM2-1.7B-Instruct
- Phi-4-mini-instruct
- Falcon3-7B-Instruct

Tokenization:

```
['The', 'Ġquick', 'Ġbrown', 'Ġfox', 'Ġjumps',
 'Ġover', 'Ġthe', 'Ġlazy', 'Ġdog', '.']
```

These models produced an identical tokenization consisting of 10 tokens. The Ġ prefix indicates that the token is preceded by a space, a convention commonly used in GPT-style BPE tokenizers. Both "fox" and "jumps" appear as complete vocabulary entries and are represented by single tokens.

Group 2 Models:

- Mistral-7B-Instruct-v0.3
- DictaLM2.0-Instruct

Tokenization:

```
['_The', '_quick', '_brown', '_f', 'ox',
 '_j', 'umps', '_over', '_the',
 '_lazy', '_dog', '.']
```

These models share a SentencePiece-based tokenization strategy, where the symbol marks the beginning of a word. Unlike Group 1, the words "fox" and "jumps" are split into multiple subword tokens (`f + ox` and `j + umps`), resulting in 12 tokens instead of 10.

Group 3 Models:

- Granite-3.3-8B-Instruct

Tokenization:

```
['The', 'Ġquick', 'Ġbrown', 'Ġf', 'ox',
 'Ġjumps', 'Ġover', 'Ġthe',
 'Ġlazy', 'Ġdog', '.']
```

The Granite tokenizer is similar to Group 1 and uses the same `Ġ` space-prefix convention. However, it splits the word "fox" into two subword tokens (`Ġf` and `ox`) while keeping "jumps" as a single token. As a result, the sentence is represented using 11 tokens.

Analysis The three groups demonstrate how tokenizer vocabularies and merge rules affect text segmentation. Group 1 produces the most compact representation (10 tokens), Group 2 performs the most aggressive subword splitting (12 tokens), and Granite lies between them (11 tokens). These differences highlight how tokenizer design influences token counts and vocabulary coverage, even for a simple and common English sentence.

To explicitly answer how many of the remaining 7 tokenizers agree with each of the 3 chosen tokenizer groups on this specific text:

- If we select one representative from **Group 1** (e.g., Llama-3.1-8B), **6 of the remaining 7 tokenizers** produce the exact same tokenization.
- If we select one representative from **Group 2** (e.g., Mistral-7B), **1 of the remaining 7 tokenizers** (DictaLM2.0) agrees with it.
- If we select the representative from **Group 3** (Granite-3.3-8B), exactly **0 of the remaining 7 tokenizers** agree with its specific tokenization split.

Special and Non-Word Tokens

Across the evaluated tokenizers, a significant portion of the vocabulary does not correspond to standard words or subwords. Depending on the model, there are anywhere from a few dozen to several hundred non-word tokens. These generally encode structural or functional data and fall into three distinct categories:

1. **Control and Formatting Tokens:** These are explicitly encoded special tokens used to structure the prompt, delineate conversation turns, or pad sequences. The count varies greatly depending on the model's architecture. For instance, Mistral uses roughly 3 to 5 core tokens (e.g., `<s>`, `</s>`, `<unk>`), while advanced multimodal models like Qwen allocate over 20 specific tokens (e.g., `<|im_start|>`, `<|vision_pad|>`). Furthermore, models like Llama-3 reserve a dedicated block of up to 256 tokens for special structural tags (e.g., `<|start_header_id|>`, `<|end_of_text|>`). They are represented internally by specific un-mergeable integer IDs.

2. **Whitespace and UI Elements:** Many BPE-based tokenizers allocate a few hundred tokens (typically estimated between 100 and 300) specifically to encode whitespace geometries. These represent multiple consecutive spaces (e.g., ĠĠĠ), tab indents (`\t\t`), or specialized line breaks. They are represented just like regular tokens but encode non-printable characters. They are crucial for accurately reconstructing and formatting code generation or structured documents without the model needing to predict individual spaces sequentially.

3. **Byte-Fallback Mechanism Tokens:** Several tokenizers (such as Llama-3 and Qwen) use a byte-level fallback. They reserve exactly **256 tokens** to represent raw UTF-8 bytes (e.g., `<0x00>` to `<0xFF>`). If the tokenizer encounters an unknown Unicode character (such as an obscure emoji or a rare language script not present in the vocabulary), it decomposes the character into its constituent UTF-8 bytes and represents it using these 256 non-word byte tokens. This mechanism ensures the model never encounters a hard out-of-vocabulary (OOV) error.

Constrained Decoding

Hebrew Token Identification

For each of the two models, we constructed a separate set of token ids that may participate in Hebrew text. This was necessary because token ids are tokenizer-specific: the same Hebrew string may be represented by different token ids in Qwen and in Mistral. Therefore, we submitted two separate JSON files:

```
hebrew_allowed_tokens_qwen.json
hebrew_allowed_tokens_mistral.json
```

Each file has the required structure:

```
{
  "model_id": "...",
  "allowed_token_ids": [1, 2, 3]
}
```

Our token-selection strategy was based on decoding each token in the tokenizer vocabulary and inspecting the Unicode characters in the decoded string. A token was included in the allowed set if it satisfied one of the following conditions:

1. The decoded token contained Hebrew characters.
2. The decoded token contained only neutral characters that can naturally appear in Hebrew text, such as whitespace, digits, punctuation marks, mathematical symbols, currency symbols, emoji, or other non-letter symbols.
3. Special tokenizer tokens, such as beginning-of-sequence, end-of-sequence, padding, unknown tokens, or chat/control tokens, were not included in the submitted `allowed_token_ids` JSON files. Instead, when such tokens were needed for generation control, stopping, padding, or chat-template handling, we added or handled them explicitly in the generation code.

Thus, the submitted JSON files intentionally contain only the Hebrew-compatible and neutral text tokens. Special tokens were treated as implementation-level generation controls rather than as part of the Hebrew text vocabulary itself.

The Hebrew Unicode ranges we used were:

```
U+0590--U+05FF
U+FB1D--U+FB4F
```

The first range includes Hebrew letters, Hebrew punctuation, cantillation marks, and niqqud. The second range includes Hebrew presentation forms. Thus, Hebrew vowel marks and related Hebrew marks were also allowed.

The main exclusion rule was that tokens containing non-Hebrew alphabetic characters were not allowed. For example, tokens containing English, Chinese, Arabic, Korean, or other non-Hebrew letters were excluded. However, digits, punctuation, emoji, and general symbols were kept, since they may legitimately appear in Hebrew text.

This strategy is intentionally simple and token-based. It does not attempt to determine whether a full word is meaningful Hebrew, nor does it check grammar or semantics. It only determines whether the decoded token is compatible with Hebrew text according to character-level Unicode rules. This matches the goal of the assignment, since the constrained decoder operates over token ids during generation.

Constrained Decoding Implementation

We implemented constrained decoding by adding a custom Hugging Face `LogitsProcessor` to the generation procedure.

At every generation step, the language model produces a score for every token in the vocabulary. Our constrained decoder receives these scores and assigns $-\infty$ to every token that is not in the allowed Hebrew-compatible token set. As a result, the model can only generate tokens from the allowed set.

Conceptually, if V is the full vocabulary and $A \subseteq V$ is the allowed Hebrew-compatible token set, then for every token $t \in V$ we replace the original score s_t with:

$$s'_t = \begin{cases} s_t, & t \in A \\ -\infty, & t \notin A \end{cases}$$

This guarantees that disallowed tokens receive zero probability after the softmax operation.

In code, this was implemented using a class similar to the following:

```
class AllowOnlyTokensLogitsProcessor(LogitsProcessor):
    def __init__(self, allowed_token_ids):
        self.allowed_token_ids = set(int(t) for t in allowed_token_ids)

    def __call__(self, input_ids, scores):
        vocab_size = scores.shape[-1]
        mask = torch.full(
            (vocab_size,),
            -float("inf"),
            device=scores.device,
            dtype=scores.dtype,
        )
        mask[allowed_token_ids_tensor] = 0.0
        return scores + mask
```

The processor was passed to `model.generate` using:

```
logits_processor=LogitsProcessorList([
    AllowOnlyTokensLogitsProcessor(self.allowed_token_ids)
])
```

We implemented both constrained and unconstrained generation inside the same inference class. In both cases, we used the tokenizer's chat template:

```
tokenizer.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt",
    return_dict=True,
)
```

This is important because Qwen and Mistral use different instruction formats. Qwen uses its own chat-template structure with explicit system/user/assistant turns, while Mistral uses an `[INST] ... [/INST]` style format. By using `apply_chat_template`, we avoided manually writing model-specific prompts.

The only difference between the two decoding modes was therefore:

- **Unconstrained decoding:** standard `model.generate`.
- **Constrained decoding:** `model.generate` with the Hebrew-token `LogitsProcessor`.

Prompt and Prefix Experiments

Before running the final decoding comparison, we experimented with different ways of encouraging the models to answer in Hebrew. The goal of these preliminary experiments was to understand the interaction between two separate mechanisms:

1. prompting the model to answer in Hebrew;
2. forcing the model to generate only Hebrew-compatible tokens.

These two mechanisms are not equivalent. A Hebrew instruction changes the model's probability distribution and encourages Hebrew output. The constrained decoder only blocks illegal tokens; it does not by itself make the model understand that it should answer naturally in Hebrew.

In preliminary tests, we tried variants such as:

- no Hebrew instruction;
- a Hebrew prefix such as "ענה בעברית``;
- an English prefix such as "answer only in Hebrew and check your output";
- a system message such as "ענה רק בעברית``.

However, in the final run used for the submitted output files, we did not use a different prefix for each model. Instead, we used the prefix that appears in the final code, namely:

```
answer only in Hebrew and check your output
```

followed by the English question.

The preliminary prefix experiments were still useful. When no Hebrew instruction was given, the constrained decoder often produced extremely poor output, sometimes consisting mostly of punctuation or repeated fragments. for e.g

```
=== CHAT TEMPLATE PROMPT ===
<s>[INST] Explain why we need sleep.[/INST]

    assistant:
1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.1.
```

This happened because the model was still internally trying to continue in English, but the English tokens were blocked by the constraint. Therefore, the model had to choose from the remaining Hebrew-compatible tokens even though those tokens were not necessarily likely to form a meaningful answer.

This shows that constrained decoding alone is not enough. The model should also receive a prompt that makes Hebrew a likely continuation.

We also observed a difference between the two chat templates. In Mistral, the system instruction is not necessarily visible as a separate system block in the rendered prompt. Instead, it is merged into the `[INST] ... [/INST]` instruction text. For example, a system instruction and user query may be rendered approximately as:

```
<s>[INST]       .

explain briefly what is supervised learning [/INST]
```

This differs from Qwen, where the chat template more explicitly represents separate system, user, and assistant turns. This difference further supports the decision to use `apply_chat_template` rather than manually formatting the prompt.

Decoding Output Files

We ran 10 English-language queries for each model. For every query and every model, we saved:

1. the original English prompt;
2. the model name;
3. the unconstrained output;
4. the constrained output.

The required raw output file was saved as:

`decoding_outputs.jsonl`

Each line is a separate JSON object of the following form:

```
{"prompt":"...", "model":"...", "unconstrained_output":"...",
 "constrained_output":"..."}
```

We also saved a CSV version for easier inspection and comparison, with the columns:

`model_id, question, constrained, unconstrained`

Discussion of Results

The constrained decoder successfully enforced the token-level restriction. In the constrained outputs, the models were prevented from generating ordinary English words and other foreign-language alphabetic tokens. This confirms that the `LogitsProcessor` worked as intended: disallowed tokens were blocked at every generation step.

However, the quality of the Hebrew varied significantly between the two models and between constrained and unconstrained decoding.

Unconstrained Outputs

In the unconstrained setting, the models were free to generate any token in their vocabulary. Even though the prompt asked for Hebrew, the models did not always obey this instruction reliably.

Qwen sometimes produced answers that were partially Hebrew but also contained English fragments, Chinese fragments, or other non-Hebrew text. This suggests that Qwen has strong multilingual capacity, but without a hard decoding constraint it may switch languages or include unwanted multilingual artifacts.

Mistral also failed to consistently produce clean Hebrew in the unconstrained setting. Some answers began in Hebrew but then switched to English, or mixed Hebrew with English explanatory text. Thus, for both models, prompting alone was not sufficient to guarantee Hebrew-only output.

0.2.1 Constrained Outputs

In the constrained setting, English and other foreign-language alphabetic tokens were successfully blocked. This made the outputs more language-controlled: they generally contained only Hebrew-compatible characters, punctuation, numbers, and symbols.

Nevertheless, constrained decoding did not guarantee good Hebrew. The Hebrew quality was often lower than in natural human-written Hebrew. We observed several kinds of problems:

- unnatural phrasing;
- incorrect word choices;
- repeated words or repeated sentence structures;
- semantically weak or partially unrelated explanations;
- malformed Hebrew sentences;
- cases where the answer was Hebrew-compatible at the token level but not fluent Hebrew.

This is an important distinction: the constraint controls the token set, not the linguistic quality. A generated output can satisfy the token constraint while still being bad Hebrew.

Hebrew Quality

The quality of Hebrew was one of the clearest differences between the models.

Qwen generally produced better constrained Hebrew than Mistral. Its constrained answers were more often recognizable as Hebrew explanations and were more often related to the original question. For simple explanatory questions, Qwen frequently generated content that at least mentioned the relevant concepts. For example, in questions about natural phenomena, the constrained output often included relevant Hebrew words related to light, water, the moon, the sun, or the environment.

However, Qwen's Hebrew was still imperfect. We observed mistranslations, awkward constructions, and cases where the answer was only partially semantically related to the question. In some cases, the model seemed to choose Hebrew tokens that were locally plausible but globally unnatural. Therefore, constrained Qwen output was usually more Hebrew-controlled, but not always fluent or fully correct.

Mistral's constrained Hebrew was generally weaker. The outputs were often more repetitive, less natural, and less semantically coherent. In several cases, the model produced Hebrew-compatible text that looked like a distorted template rather than a natural answer. This suggests that once English tokens were blocked, Mistral had more difficulty finding a fluent Hebrew continuation.

The difference between the two models suggests that constrained decoding depends strongly on the model's underlying Hebrew ability. If a model already assigns high probability to good Hebrew continuations, the constraint can help enforce clean Hebrew output. If the model does not naturally produce strong Hebrew, the constraint may only force it into the Hebrew token subset without producing fluent language.

Relation to the Questions

The constrained outputs were not always equally related to the English questions. Qwen was usually more successful at preserving the topic of the question under the Hebrew constraint. Mistral more often produced outputs that were only weakly connected to the prompt.

This again shows that constrained decoding is not a semantic control method. It can block disallowed languages, but it does not guarantee that the model answers the question correctly. The model still needs to understand the English prompt and map it to a Hebrew answer.

Languages Favored by the Models

The unconstrained results suggest that the models may favor different languages or switch between languages depending on their pretraining and instruction tuning. Qwen, being a highly multilingual model, sometimes produced multilingual artifacts when unconstrained. Mistral tended to produce poorer Hebrew and sometimes switched back to English.

The constrained decoder removed this multilingual leakage by making non-Hebrew alphabetic tokens impossible to generate. However, removing leakage is not the same as improving fluency. In some cases, blocking the model's preferred English continuation caused the model to produce awkward Hebrew or punctuation-heavy text.

Examples of Constrained Decoding Quality

Table **??** shows representative examples from the constrained decoding outputs. We manually grouped the examples into three qualitative categories: good or mostly correct answers, partially correct answers with weak Hebrew, and outputs that are mostly gibberish or unrelated to the question.

Model	Prompt	Constrained output excerpt	Assessment
Qwen	Explain why people recycle paper and plastic.	``אנשים מטילים את הנייר והפלסטיק שוב כדי להימנע מהפרעה לסביבה ולהגן על הסביבה... ליצור מוצרים חדשים באמצעות חומרים שכבר קיימים..."	Mostly correct. The Hebrew is awkward, especially "מטילים`` instead of "ממחזרים``, but the answer is clearly related to recycling and environmental protection.
Qwen	Explain why the ocean has waves.	``הים מופלא בגלים שלו, וגלים נוצרים בשלושה גורמים עיקריים: הרוח... רעידות האדמה או צונאמי..."	Mostly correct. The Hebrew is not fully natural, but the answer mentions wind and other wave-generating factors, so it is semantically related to the question.
Qwen	Explain why rainbows appear after rain.	``...אחרי גשם... השמש... המים... מפרקים את האור שלה... צבעים..."	Partially correct. The answer captures the basic idea of sunlight interacting with water droplets and splitting into colors, but contains wrong word choices such as "ריבועים`` instead of "קשתות`` and some unnatural phrasing.
Qwen	Explain why we need sleep.	``הישן הוא חובה חיוני עבור הגוף והמוח שלנו... במהלך השינה... הגוף... המוח..."	Partially correct. The topic is preserved and the answer discusses the body and brain, but the Hebrew is weak: "הישן`` and "צריכים לשון`` are incorrect or unnatural.
Qwen	Explain why brushing your teeth is important.	``החישוב של שיניים הוא חשוב... בריאות השיניים והפה... אסתטיקה..."	Weak but somewhat related. The model clearly refers to teeth and oral health, but uses the wrong word "חישוב`` instead of "צחצוח``, making the Hebrew quality poor.
Qwen	Explain why traffic lights are useful.	``הצבעות האוטובוסיות הן שימושיות... לנהגים... בטיחות על הכבישים..."	Partially related but lexically broken. The answer discusses drivers, roads, and safety, but "הצבעות האוטובוסיות`` is an incorrect rendering of traffic lights.

Model	Prompt	Constrained output excerpt	Assessment
Mistral	Explain why rainbows appear after rain.	``הריוונים מצאים אותם... מים... אור... צבעים..."	Weak but partially related. The output mentions water, light, and colors, but the Hebrew is highly distorted and repetitive.
Mistral	Explain why people recycle paper and plastic.	``האנשים מרכזים את הכרטון ו הפלסטיק... לא לשלום האדמה..."	Weak but somewhat related. The output mentions paper/cardboard, plastic, and the ground/environment, but the wording is unnatural and the explanation is not fluent.
Mistral	Explain why the moon changes shape during the month.	``אני, איתאי... מעבירתך אתה למיקום שמונה של הלול... הלול משתנה..."	Mostly gibberish. The output is Hebrew-compatible at the token level, but it is not a meaningful explanation of moon phases.
Mistral	Explain why traffic lights are useful.	``האורגניותים... האלכטרית... מסעים רשימיים... מרכזים ומסעדות..."	Gibberish / unrelated. The generated text contains Hebrew-like words and punctuation, but it does not meaningfully answer the question.
Mistral	Explain why brushing your teeth is important.	``מחשיך שאריתך משמעותית... המחשיך משמעותית... השאריתך המשמעותית..."	Gibberish / very low quality. The output is repetitive and does not provide a meaningful explanation about brushing teeth.

Table 3: Representative examples of constrained decoding output quality. The examples show that constrained decoding successfully blocks non-Hebrew alphabetic tokens, but does not guarantee fluent or semantically correct Hebrew.

The examples show a clear difference between token-level validity and language quality. In almost all constrained examples, the models obey the hard restriction and avoid ordinary English tokens. However, the Hebrew quality varies substantially. Qwen usually produces outputs that are at least related to the question, even when the Hebrew is awkward or contains mistranslations. Mistral, in contrast, often produces text that is Hebrew-compatible at the character/token level but semantically weak, repetitive, or almost meaningless.

This supports our main conclusion: constrained decoding is effective as a hard language filter, but it is not a guarantee of good Hebrew. A model may be forced to generate only Hebrew-compatible tokens and still produce unnatural Hebrew, wrong word choices, or gibberish. The constraint prevents language leakage; it does not solve fluency, factuality, or semantic alignment.

Conclusion

The experiment shows that simple token-level constrained decoding can successfully enforce a Hebrew-compatible output space. The implementation is straightforward: we identify a set of allowed Hebrew-compatible token ids, handle the tokenizer's special tokens separately when needed for generation control, and use a `LogitsProcessor` to assign $-\infty$ to every disallowed token during generation.

The main conclusion is that constrained decoding is effective for hard language filtering, but it is not sufficient for guaranteeing high-quality Hebrew. The semantic and linguistic quality of the output still depends on the model's Hebrew capability and on the prompt.

Qwen performed better than Mistral in this experiment: its constrained outputs were generally more related to the questions and more recognizable as Hebrew explanations. Mistral's constrained outputs were more often repetitive, unnatural, or semantically weak. Still, both models showed that constrained decoding can prevent unwanted English or multilingual output.

The best results came from combining both methods: an explicit instruction asking for Hebrew, together with constrained decoding that enforces the Hebrew-compatible token set.

Fine Tuning

The fine-tuning partially worked. It successfully shifted the model's output distribution toward Hebrew and taught it to mimic assistant-style Hebrew responses. However, it did not fully preserve semantic correctness, and on out-of-distribution prompts the model often produced hallucinations, unnatural Hebrew, and cross-lingual token bleeding. Therefore, the method worked as a language-shift intervention, but not as a fully reliable Hebrew assistant.

Data Preparation and Experimental Setup

Adapting a compact, 1.5B parameter instruction-tuned model to perform cross-lingual behavioral shifts (English instructions $\rightarrow$ Hebrew responses) is a delicate balancing act. To optimize our training pipeline, we systematically designed, implemented, and tested three distinct data preparation paradigms, tracking how the underlying distribution of the data governed the model's downstream capabilities.

Data Sourcing and Initial Engineering Pipelines

Our initial strategy focused on utilizing data sourced from both curated human-aligned corpora and structured synthetic environments to observe how the model responded to isolated linguistic properties.

- **Transformed Open-Source Corpus (Alpaca Hebrew TACO):** We targeted the open-source `saillab/alpaca_hebrew_taco` dataset on Hugging Face. Because our target task requires handling English user instructions, we built an engineering pipeline to invert the language directional flow. For each row, we combined the original `instruction` and `input` columns, translated them into English to form the input training prompt, and maintained the original Hebrew `output` as the target training response.
- **Synthetic LLM-Generated Dataset:** To complement the foundational data with everyday conversational elements, we utilized a synthetic corpus of 5,000 rows containing general-purpose instruction-response vectors. This dataset provided formulaic responses to standard user intents.

Data Quality Control Note: Both source datasets were highly scrubbed and strictly verified. Programmatic character validation confirmed that the target responses contained pure Hebrew text, completely free of foreign characters, English words, or pre-training artifacts. the prompt given to the llm:

צור לי 5000 שאלות שונות מהשאלות האלו:

- Explain why the sky looks blue during the day.
- Give two advantages and two disadvantages of public transportation.
- Write a short email asking a professor for an extension on an assignment.
- Describe how to make a simple omelette.
- What is the difference between supervised and unsupervised learning?
- Summarize the story of Cinderella in three sentences.
- Suggest three ways to reduce smartphone distraction while studying.
- Explain what happens when water boils.
- Give a polite refusal to an invitation to a party.
- Turn the idea "practice makes progress" into advice for a student.

אבל באותו סגנון (אסור פרפרזה של השאלות האלו) ותשובות בעברית.

צור קובץ jsonl בפורמט הבא:

```
{"instruction": "Turn this sentence into a question: The man
was wearing a hat.", "response_hebrew": "האם האיש חבש כובע?"}
```

תבדוק צעד צעד שאכן התשובות הן בעברית נכונה והן אכן עונות על השאלות, שיש תמהיל טוב לסוגי השאלות, ושהפורמט נכון ורק אז תיתן לי לינק להוריד את הקובץ.

Three-Model Experimental Configurations

To find the optimal tuning threshold, we split our data engineering into three explicit model training pipelines:

1. **Model 1 (Combined Corpus):** We concatenated the clean subsets of both the translated HuggingFace corpus and the synthetic LLM dataset into a single, merged training pool. We applied a formatting map to standardize the rows into a unified ChatML conversational layout. This configuration combined the structural semantic rigor of the Alpaca dataset with the conversational flexibility of the synthetic data.

2. **Model 2 (HuggingFace Only):** To measure the true downstream utility of the inverted open-source alignment data on its own, Model 2 was trained exclusively on the transformed HuggingFace dataset, completely isolating it from any supplemental synthetic LLM data.

3. **Model 3 (Cross-Lingual Self-Distillation):** We recognized that forcing a 1.5B model to simultaneously learn a strict translation mapping while forcing it to memorize completely new, external facts often overloads its parameter capacity. To bypass this bottleneck, we extracted the complete prompt set (the combined instructions from both datasets) and ran them directly through the *base, un-tuned* Qwen2.5-1.5B-Instruct model using greedy decoding (`do_sample=False`) to capture its native, pre-trained English reasoning paths. We then passed these English outputs through a Google Translation utility via the `deep-translator` library to map them into fluent Hebrew, isolating the fine-tuning variable strictly to structural vocabulary alignment.

Fine-Tuning Execution and Hyperparameter Optimization

Phase 1: Sanity Check (Micro-Batch Overfitting)

Before initiating the full training phase, we conducted a standard machine learning sanity check by intentionally overfitting the model on a micro-batch of 20 samples. The goal was to verify that the loss function would converge and that the model could physically learn the target mapping. Evaluation on this overfit set showed perfect reconstruction of the target Hebrew text. This confirmed that our data pipeline, ChatML tokenization mapping, and backpropagation loop were functioning correctly without formatting errors.

Phase 2: QLoRA Setup and Hyperparameter Search

To train the Qwen2.5-1.5B-Instruct model within the VRAM constraints of our hardware, we utilized QLoRA (Quantized Low-Rank Adaptation). We loaded the base model using a 4-bit NormalFloat (`nf4`) quantization via `BitsAndBytesConfig`, enabling double quantization and utilizing `bfloat16` compute types to maximize memory efficiency without sacrificing gradient stability.

Rather than relying on default parameters, we integrated `Optuna` and `Weights & Biases (WandB)` to perform an automated hyperparameter search aimed at minimizing the `eval_loss`. Our search space explored LoRA Rank ($r \in \{4, 8, 16, 32\}$), LoRA Alpha ($\alpha \in \{8, 16, 32\}$), Dropout ($0.05 - 0.30$), and learning rate parameters.

Phase 3: Final Configuration and Theoretical Insights

The Optuna optimization concluded with the following optimal hyperparameters: $r = 16$, $\alpha = 32$, Dropout $= 0.05$, a learning rate of $\approx 3.48 \times 10^{-5}$, and a warmup ratio of ≈ 0.033.

These parameters provided valuable theoretical insights into the mechanics of cross-lingual adaptation:

1. **The Necessity of a Higher Rank ($r = 16$):** While simple stylistic fine-tuning tasks can often be achieved with a rank of 4 or 8, completely shifting a model's output language distribution requires significantly more parameter capacity. A higher rank ($r = 16$) allowed a larger percentage of the base model's weight matrices to be adjusted, providing the necessary dimensional space to map English instructions to Hebrew tokens.

2. **Low Dropout (0.05):** A heavily reduced dropout rate is architecturally logical for LoRA. Because Parameter-Efficient Fine-Tuning only updates a minuscule fraction of the total model weights, applying aggressive dropout severely disrupts the already constrained learning signal.

Notably, Model 3's self-distillation method caused the model to converge significantly faster. While previous iterations required approximately 1,000 steps to reach optimal loss, the self-distillation method began to overfit after only 100–150 steps while maintaining superior semantic coherence.

For the final Supervised Fine-Tuning (`SFTTrainer`) run, we applied the LoRA adapters to all linear layers within the attention and MLP blocks to maximize expressiveness. The model was trained using the `adamw_torch` optimizer, a cosine learning rate scheduler, a batch size of 2, and 8 gradient accumulation steps, utilizing gradient checkpointing to maintain VRAM stability throughout the 150-step training cycle.

Evaluation Methodology: A Dual-Assessment Approach

To ensure a comprehensive and rigorous assessment of the fine-tuned models, we employed a dual-evaluation strategy combining scalable automated metrics with qualitative human inspection.

1. Automated Evaluation: LLM-as-a-Judge

Given the complexity of evaluating cross-lingual semantic alignment and generative fluency, standard lexical metrics (such as BLEU or ROUGE) are often insufficient, as they fail to capture logical coherence. Instead, we deployed an external **LLM-as-a-Judge** framework.

We generated answers from all three model variations for a standardized set of 10 baseline queries. The judge model was tasked with blindly scoring the outputs on a scale of 1.0 to 5.0 based on four primary criteria: Semantic Alignment, Grammatical Fluency, Instruction Adherence, and Vocabulary Purity.

2. Qualitative Human Evaluation: Held-Out Test Set

While the LLM-as-a-Judge protocol provides an excellent standardized baseline, automated judges can occasionally overlook subtle logical fractures or misinterpret hallucinated formatting as a high-quality response. To counter this limitation, we conducted a rigorous manual evaluation.

During our initial data preparation phase, we strictly partitioned a **held-out test set** from our custom curated dataset. This test split was completely isolated from both the training and validation pipelines to prevent any data leakage. By passing these strictly out-of-distribution prompts to the models, we were able to manually assess the true coherence, logical consistency, and behavioral stability of the generated Hebrew text. This human-in-the-loop review was critical for identifying the latent space collapse and token fragmentation anomalies that characterized the failures in our earlier iterations.

Evaluation Results

Below is the structured verdict and performance breakdown provided by the automated judge , its important to note that the score is relative to the models, whilst Model 1 was the strongest relative configuration among the three, although its absolute performance remained imperfect:

1. Model 1 (The Winner) — Score: 4.9 / 5.0

Model 1 emerged as the strongest relative configuration among the three, although its absolute performance remained imperfect.

- **Strengths:** It generally understood the core semantic intent of the English prompts better than the other variants. It proved capable of producing several highly fluent and correct answers, notably performing well on translation tasks and structured prose generation.
- **Weaknesses:** It still suffered from clunky phrasing and poor vocabulary choices in formal contexts, completely failed the mathematical logic question, and occasionally slipped English or Spanish tokens into its generation.

2. Model 3 (Runner-Up) — Score: 3.2 / 5.0

Model 3 performed marginally better than Model 2 but remained highly flawed due to severe contextual distortions. We observed that the generated outputs successfully mirrored the base model's verbosity and structural style, indicating a strong semantic shift toward Hebrew; however, the generation was ultimately corrupted by cross-lingual hallucinations.

- **Strengths:** It attempted to structure its answers cleanly, demonstrating a strong architectural preference for organized outputs, numbered lists, and bullet points.
- **Weaknesses:** It was overly verbose and frequently suffered from bizarre semantic hallucinations (e.g., misinterpreting or shifting historical references). More concerningly, it regularly leaked Chinese and Russian tokens into its Hebrew sentences.

3. Model 2 (The Worst) — Score: 3.0 / 5.0

Model 2 struggled the most with both basic English instruction comprehension and Hebrew language generation.

- **Strengths:** It successfully managed to process and translate simple, highly conversational, and formulaic baseline prompts.
- **Weaknesses:** It consistently broke character by inserting random gibberish into strings, along with Korean, Chinese, and English characters into the middle of Hebrew words. Its sentence structures were frequently nonsensical, fractured, and entirely unintelligible.

Inference-Time Observations and Latent Space Breakdown

Crucially, the experimental results reveal a sharp distinction between dataset cleanliness and model generation behaviors. While the source training corpora for all models were thoroughly filtered and linguistically pure, evaluating the downstream checkpoints on out-of-distribution prompts exposed severe structural vulnerabilities inherent to smaller architectures.

When Model 1 and Model 2 were subjected to supervised fine-tuning, they suffered from catastrophic overfitting, leading the model to memorize strict target text paths rather than learning generalized task translation. During testing, this caused its internal logic to fracture on non-memorized prompts, outputting severe semantic hallucinations.

Furthermore, because of the exceptionally compact latent space of a 1.5B parameter model, pushing the parameters too close to a flattened Hebrew distribution caused severe **cross-lingual token bleeding** during decoding. At inference time, the flattened logit probabilities allowed pre-training artifacts to leak out, forcing the model to bleed unexpected English, Korean, Japanese, or Chinese characters directly into the middle of Hebrew words due to token fragmentation inside Qwen's heavily multilingual tokenizer.

Anecdotal Input-Output Comparison

To demonstrate the effects of the fine-tuning, we selected an anecdotal input-output pair that perfectly encapsulates both the architectural successes and the latent space limitations of our fine-tuning process.

Prompt:
"What are the key benefits of learning a new language?"

Base Model Output (Unconstrained):

Learning a new language can have several key benefits:

1. Increased job opportunities: Knowing multiple languages can make you more attractive to employers and open up new career opportunities.
2. Improved communication skills: Learning a new language can improve your ability to communicate with people from different cultures and backgrounds.
3. Enhanced cultural understanding: Languages reflect the culture they are spoken in...

Fine-Tuned Model Output (LoRA):

לימוד שפה חדשה יכולה להוביל למספר יתרונות חשובים, כולל:

1. ה[Chinese :ו-הויות[Character לימוד שפה חדשה עוזר לשפר את השכלות הכללית והיכולות הרפיה שלך, כמו גם לשפר את הביצועים האנרגטיים שלך.
2. רווחה כלכלית: יושבים עם אנשים ממדינות שונות מאפשר לך לקבל החלטות יקרות יותר ולקבל הזדמנויות עבודה גדולות יותר.
3. רשת חברתי חזקה: למידת שפה חדשה עוזרת לך ליצור קשרים חזקים עם אנשים ממדינות אחרות, מה שיוצר חלוף וחיבור.
4. אינטראקציה: למידה בתרחישים אינטראקטיביים יכולות לעזור לשפר את הביצועים הקוגניטיביים של המחשבה, התנהגות ובינה מלאכותית.

Analysis and Justification

We chose this specific pair because it highlights exactly what changed—and what broke—after fine-tuning the 1.5B parameter model.

- **Structural Mimicry (Success):** The fine-tuned model actively attempts to generate results that are structurally similar to the base model, successfully learning the behavioral format (an introductory sentence followed by a numbered list). It clearly understands *how* an assistant should format an answer.
- **Language Shift (Success):** The output completely transitioned from English to Hebrew, fulfilling the primary objective of the Supervised Fine-Tuning.
- **Cross-Lingual Token Bleeding (Failure):** The most fascinating takeaway from this pair is the presence of the Chinese character "shan" (meaning "expert" or "good at") injected directly into the first bullet point. The forced flattening of the English-to-Hebrew distribution caused token fragmentation, leading the model to reach for a concept it lacked the exact Hebrew tokens for and bleeding a Chinese ideograph directly into a Hebrew word.
- **Semantic Hallucination (Failure):** While the base model gives logical, well-reasoned benefits, the fine-tuned model hallucinates bizarre, semantically distorted concepts, claiming language learning improves "energy performance" and "artificial intelligence."

This suggests that our LoRA fine-tuning was effective at teaching the model *how* to behave at the structural level, namely by shifting the output language and imitating assistant-style formatting. However, for a compact 1.5B-parameter model, the available data was not diverse enough to preserve semantic reliability. The model often traded logical reasoning for localized Hebrew syntax mimicry, which resulted in hallucinations, token bleeding, and unstable generalization.

Appendix A: Fine-Tuning 20 Evaluation Inputs

As required by the assignment, below are the 20 evaluation inputs (10 custom inputs followed by the 10 provided in the assignment prompt). For each input, we display the Base Model's original output alongside the Fine-Tuned Model's output to demonstrate the language shift and inference behavior. *Note: Minor cross-lingual artifacts generated by the model during inference have been tagged as [Foreign Character] to preserve document compilation formatting.*

English Prompt	**Base Model Output**	**Fine-Tuned Model Output**
Write a polite email apologizing to a client for missing a scheduled meeting due to a sudden internet outage.	Subject: Apology and Explanation Regarding the Recent Internet Outage. Dear [Client's Name], I hope this message finds you well. Firstly, I would like to apologize sincerely for any inconvenience or frustration caused by the recent unexpected internet outage that prevented me from attending our planned meeting as scheduled...	שלום, אני מצטער/ת שלא הצלחתי להגיע לפגישה מוקדמת. אשמח לקבוע מועד חדש אם זה עדיין מתאים.
Provide actionable steps for a manager based on the concept that 'psychological safety drives innovation' in a remote team.	To foster psychological safety and drive innovation within a remote team, managers can take several actionable steps: 1. Establish Open Communication Channels... 2. Create a Safe Space: Ensure that your team feels safe to share their ideas, feedback, and concerns without fear of retribution...	1. הגדר סביבת עבודה בטוחה: ערכו ארגון סביבתי שבו כל אחד מרגיש בטוח להביע דעה או רעיון ללא חשש מחוסר ביטחון או אי הבנות. 2. הציעו תובנה ברורה על מעורבות... 3. קדם מדיה פתוחה... 4. הגביר אמון: הגביר אמון בתוך הקבוצה...
What is the solution to this puzzle: If a car travels at a constant speed of 60 miles per hour, how many minutes will it take to travel 45 miles?	To find out how many minutes it takes... we can use the formula: Time = Distance / Speed. First, convert the distance from miles to hours... Time in hours = 45 miles / 60 mph = 0.75 hours. Now, convert the time from hours to minutes: 0.75 hours × 60...	אם רכבת נוסעת במהירות קבועה של 60 מייל לשעה, אז זמן הריצה שלו ל-45 מייל הוא 45 מייל / 60 מייל לשעה = 0.75 שעות. כדי להמיר שעות למינוחים, כותבים 0.75 שעות × 60 דקות לשעה = 45 דקות. לכן, זמן הנסיעה של 45 מייל יהיה 45 דקות.
Translate 'Thank you for your quick response and assistance' into hebrew.	Sure! The translation of "Thank you for your quick response and assistance" into Hebrew is: . This can be broken down as: (Toda) - Thank you...	"תודה על התגובה מהירה והעזרה שלך".

English Prompt	Base Model Output	Fine-Tuned Model Output
Explain in simple terms how a microwave oven actually heats up food.	A microwave oven uses electromagnetic waves to heat your food. Here's a simple explanation: 1. Microwave Waves: Inside the oven, there are special electronic circuits that generate microwaves... 2. Wave Interaction with Water: The water molecules in your food absorb these microwaves...	ov[Foreign בך[Character מזיקה משתמש במטען gamma לאטמוספרה של רוקולוזן בתוך הרקע של המכשיר כדי לספק אורגניזמים טעימים חום. האור גורם לטמפרטורה לעלייה תוך שימור אנרגיית החום בחלקיקות קטנות יותר.
Suggest a realistic approach for this question: How can a student overcome severe procrastination when starting a massive research paper?	Overcoming severe procrastination while tackling a massive research paper is a common challenge... 1. Set Clear Goals: Break down your research paper into manageable sections... 2. Create a Timeline... 3. Use Tools... 4. Start Early... 5. Eliminate Distractions...	פתרון פוטנציאלי אחד לאתגר של סיכום עתידי בערימות גדולות יכול לכלול השלבים הבאים: 1. הגדר תאריך מוגדר ומנוצל להשלמת העבודה... 2. פירוק העבודה לחלקים קטנים יותר... 3. צור מערכת יחסים עם מחבר או מורה...
Rewrite the following sentence to sound more optimistic and constructive. Input: The project is probably going to be delayed again because our team is too small and lacks resources.	The project might still proceed on schedule if we can find ways to increase our team size and allocate additional resources.	הפרויקט עשוי להתקדם בקצב טוב יותר אם ייקח את הכוחות והמשאבים הנדרשים כדי לספק תמיכה למאמצי הקמתו.
Describe the environmental and economic impact of the transition to electric vehicles in Europe.	The transition to electric vehicles (EVs) in Europe is having significant environmental and economic impacts. Environmental Impact: 1. Reduced Greenhouse Gas Emissions... 2. Energy Savings... 3. Cleaner Air Quality... Economic Impact: 1. Job Creation...	שינוית מכוניות חשמליות בעלות בריטניה היא תהליך חיוני להפחתת פליטת גזי חממה ולמנוע שינויי אקלים ברחבי העולם. אחד ההשפעות הכלכליות הגדולים ביותר של שינוי זה הוא על תעשיות המכוניות...
Evaluate the validity of the claim presented. Input: Claim: Humans only use 10% of their brains.	The claim "Humans only use 10% of their brains" is not accurate. Research has shown that humans use almost all areas of the brain throughout our lives. While some parts of the brain may be more active than others depending on what we're doing... no part remains completely inactive.	הטענה "בני אדם משתמשים רק ב-10% מהמוח שלהם" היא שגויה. מחשבות עקביות ומחקר סטטיסטי מראים שלמעשה, כוח המחשבה והפעולות שלנו מגיעים לרוב לשניהם... אינטראקטיביות מדעיות כמו אלה עלולות להשתנות...

English Prompt	Base Model Output	Fine-Tuned Model Output
Is it beneficial for high schools to make personal finance and budgeting a mandatory graduation requirement?	Making personal finance and budgeting a mandatory graduation requirement in high school could be seen as beneficial from several perspectives: 1. Financial Literacy... 2. Budgeting Skills... 3. Career Preparation... 4. Critical Thinking...	כן, זה יכול להיות מועיל למשבטים להציע ביטוי של ניהול כספי אישי ותכנון חשבונות כחובה סופית. ישנן מספר סיבות מדוע זה יכול להיות שימושי: 1. ידע בכסף: ת[Foreign Characters] הידע והניסיון בכיסוף... 2. איזון בין משאבים... 3. ניהול רווחים...
Explain why the sky looks blue during the day.	The sky appears blue during the day due to a phenomenon called Rayleigh scattering. This is caused by the interaction between sunlight and molecules in Earth's atmosphere. When sunlight enters our atmosphere from the sun, it travels through several layers of air...	השמיים נראים כחולים כי אור הוא מונע דרך העננים. כאשר אור מגיע אל העיניים, חלק מהאור עובר דרך עור השפה ונראה כחולים.
Give two advantages and two disadvantages of public transportation.	Advantages: 1. Cost-effective: Public transportation can be more affordable... 2. Environmental benefits: Using public transportation instead of private vehicles can reduce traffic congestion... Disadvantages: 1. Limited accessibility... 2. Less flexibility...	שני יתרונות הם בודקנה וחיסכון בזמן. שני חסרונות הם tickets ומסכים מעורבים.
Write a short email asking a professor for an extension on an assignment.	Subject: Request for Extension on Assignment Submission. Dear Professor [Name], I am writing to request an extension on the submission of my assignment titled "Project Analysis"... unfortunately, unforeseen circumstances have arisen which have prevented me from completing the project as planned...	פרופסור/ה יקר/ה, אני מבקש/ת הסיבה למשהו ששהקתי מועד. האם תוכל/י בבקשה להרפות את המבחן/ה או להסיר את ה-timestamp אם אפשר? תודה רבה.
Describe how to make a simple omelette.	To make a simple omelette, follow these steps: 1. Crack 2 eggs into a bowl and beat them with a fork... 2. Heat a non-stick skillet over medium heat... 3. Pour the beaten egg mixture into the hot skillet... 4. Using a spatula, carefully fold the edges...	כרוס עוף הוא טכניקה פשוטה וקלית להכנת ארוחת ערב או ארוחת צהריים. הנה מתכון פשוט שתוכלו לנסות בבית: מרכיבים: - 2 בצלים גדולים - 1 כף חמאה קצוצה... הוראות שימוש: 1. מחממים את התנור ל-350 מעלות פרנהייט... 2. מניחים בצלים על אש בינונית...

English Prompt	Base Model Output	Fine-Tuned Model Output
What is the difference between supervised and unsupervised learning?	Supervised and unsupervised learning are two main types of machine learning algorithms... 1. Supervised Learning: This type of learning involves training a model on labeled data... 2. Unsupervised Learning: Unlike supervised learning, which requires labeled data, unsupervised learning deals with unlabeled data...	למידת עקביות ולמידת[Foreign ית[Characters הן שתי תחומים מרכזיים בלמידה מסודרת, אך יש להם כמה מהרבות ביניהם. למידה עקיפה היא תהליך של למידה של אלגוריתם על נתונים המדויקים... לעומת זאת, למידה[Foreign ית[Characters היא תהליך של למידה של אלגוריתם על נתונים שאינם מיועדים...
Summarize the story of Cinderella in three sentences.	Cinderella is a classic fairy tale about a young girl who lives with her cruel stepmother and stepsisters but eventually finds love and happiness through magic and hard work. She wears beautiful glass slippers that lead to the prince's ball... The prince discovers she is a princess and they live happily ever after.	דיהCSR_ בת בגדים רעילים מבקשת חתירה לאחר מוות אביה. היא עוזבת מקרה ל($__) ל הperformed ול[Foreign Characters]. בסוף היא ניצלת את הרוגזים והופכת למלך וספרת ימיה.
Suggest three ways to reduce smartphone distraction while studying.	1. Use noise-cancelling headphones: This can help block out distracting noises... 2. Turn off notifications: Notifications from social media or other apps can be incredibly distracting... 3. Set specific times for checking emails or social media...	אפשר להפחית ספק מטל Booth על ידי[Foreign ון[Character למשימה אחת בכל רגע, הימנעות משימוש בתחבורה או במזג אוויר גרוע, והשתמש במקלט או בשיחה עם אדם שמתמקד בזמן הלמידה.
Explain what happens when water boils.	When water boils, it changes from a liquid to a gas state. This process is called boiling and occurs at a specific temperature known as the boiling point of water. At sea level, the boiling point of water is 100 degrees Celsius...	כאשר מים בזון, הם סופגים אנרגיה ומורידים חום. התוצאה היא שחרור של זרמי גז שרירים ו-revisים את המים.
Give a polite refusal to an invitation to a party.	Dear [Name], Thank you for inviting me to your party, but unfortunately I am unable to attend. I hope you have a great time and look forward to seeing you soon. Best regards, [Your Name]	אני מצטער/ת, אבל איני יכול/ה להשתלב בפסטיבל היום. מקווה שתבין/י.

English Prompt	Base Model Output	Fine-Tuned Model Output
Turn the idea ‘practice makes progress’ into advice for a student.	”Practice is the key to progress in learning and studying. By consistently applying yourself to your studies or practice problems, you will gradually see improvements and achieve success.”	אל תפחד מהתמודדות עם חידושים קטנים. תרגול קבוע הופך את החידוש לשיפור קבוע.

Conclusion

This report systematically explored the fundamental components of modern Large Language Models, spanning base architectural design, tokenizer mechanics, inference-time decoding constraints, and parameter-efficient fine-tuning (PEFT). By examining and manipulating these layers, we gained valuable insights into how compact models process, translate, and generate language.

Our architectural analysis indicated that while the dense, decoder-only Transformer with Grouped-Query Attention (GQA) and SwiGLU activations has become standard for 7B–8B models, tokenizer design and vocabulary scale heavily influence a model's multilingual efficiency. The tokenization experiments demonstrated that models trained primarily on English-centric corpora fragment Hebrew text into significantly more subword units, leading to less efficient sequence generation.

Through our constrained decoding experiments, we showed that applying a `LogitsProcessor` serves as an effective hard filter to prevent foreign language leakage. However, this token-level restriction also highlighted a key limitation: forcing a model to generate Hebrew-compatible tokens does not inherently endow it with Hebrew fluency or semantic understanding. When constrained, models with weaker native Hebrew capabilities resorted to generating unnatural phrasing or repetitive artifacts.

Finally, our fine-tuning experiments on the Qwen2.5-1.5B-Instruct architecture highlighted the delicate balance required when inducing cross-lingual behavioral shifts in compact models. While QLoRA with optimized hyperparameters ($r = 16$, low dropout) was highly effective for teaching the model new structural formats and language translation tasks, we observed that pushing the parameters too close to a flattened target distribution can lead to severe overfitting. The resulting representational distortion triggered semantic hallucinations and "cross-lingual token bleeding," where the multilingual tokenizer leaked pre-training artifacts (such as Chinese or Russian characters) directly into the Hebrew generation.

Ultimately, achieving robust cross-lingual adaptation in compact language models cannot rely on a single technique. It requires a balanced approach involving appropriate architectural choices, an expansive and adaptable tokenizer, strategic inference-time decoding constraints, and highly curated, self-distilled training data.